

Stock Portfolio Allocation & Risk Management using Reinforcement Learning

TEAM MEMBERS

JOBINA JOY (C0924759)

AKHIL AUGUSTINE (C0924980)

ABHISHEK MUNGATH (C0928517)

TARUN THOMAS SAMUEL (C0928591)

JEFFIN KOCHURANI RAVI (C0925106)

NIKHIL PREETHA MURALI (C0928366)

ANAND CHATHANANICKAL SAJEEVAN (C0928396)

RIYO SEBASTIAN PALAKUNNEL SEBASTIAN (C0928670)

RAJKARAN YASODHADEVI PONNUVELUSAMY (C0928356)

to

Professor Bhavik Gandhi

Lambton College in Mississauga



INTRODUCTION

This project delivers an advanced framework for optimal portfolio management focused exclusively on the Toronto Stock Exchange (TSX) through state-of-the-art reinforcement learning techniques. By leveraging a comprehensive dataset of 4,066,965 historical data points spanning 1,631 potential securities, our system implements the Deep Deterministic Policy Gradient (DDPG3) algorithm for both strategic portfolio allocation and sophisticated risk management. The research meticulously processed and analyzed 1,537 stocks with sufficient historical data, applying advanced feature engineering techniques that capture nuanced market dynamics through technical indicators including Returns, Volume Moving Average, Price Moving Average, Relative Strength Index (RSI), Moving Average Convergence Divergence (MACD), Volatility, and Momentum. The integration of allocation and risk management within a single algorithm creates a robust trading system specifically optimized for Canadian market conditions, dynamically balancing profit maximization with risk minimization. This innovative approach goes beyond traditional portfolio management strategies by employing machine learning to adaptively respond to the complex and ever-changing landscape of the Toronto Stock Exchange. The tool represents a significant technological advancement for investors seeking enhanced returns while maintaining disciplined risk management, bridging cutting-edge reinforcement learning techniques with practical financial decision-making.

INDUSTRY

The stock market serves as a vital platform where investors can purchase and sell shares of publicly traded companies, acquiring partial ownership and generating returns through dividends or capital appreciation. This project specifically targets the Toronto Stock Exchange (TSX), Canada's premier equity market and a significant global financial hub. Ranked as the 10th largest exchange worldwide and 3rd largest in North America by market capitalization, the TSX represents a substantial yet often underutilized opportunity for algorithmic trading strategies.

As of January 2024, the TSX hosted 1,811 listed companies spanning diverse sectors of the Canadian economy. Our comprehensive dataset encompasses 1,631 of these listings, providing broad market coverage for our reinforcement learning models. The exchange features prominent companies across key sectors including energy, financial services, industrials, and natural resources that form the backbone of the Canadian economy.

Key TSX Components in Our Dataset:

Ticker	Company Name	Sector	Industry
CNQ.TO	Canadian Natural Resources	Energy	Oil & Gas E&P
BMO.TO	Bank of Montreal	Financial Services	Banks - Diversified
CM.TO	Canadian Imperial Bank of Commerce	Financial Services	Banks - Diversified
CNR.TO	Canadian National Railway Co.	Industrials	Railroads
ABX.TO	Barrick Gold Corporation	Basic Materials	Gold
BNS.TO	Bank of Nova Scotia	Financial Services	Banks - Diversified

DEFINING THE PROBLEMS

Individual investors face numerous challenges when managing stock portfolios that sophisticated institutional investors can overcome with advanced technologies and expertise.

Knowledge & Expertise Gap

Most retail investors possess limited understanding of the statistical methods required for true portfolio optimization. They struggle to build complex risk models, lack expertise in advanced technical analysis, and find it difficult to incorporate broader macroeconomic factors into their investment decisions.

Information Overload

Individual investors today face an overwhelming deluge of financial information that makes effective portfolio management exceedingly difficult. The modern investor must somehow process data on thousands of potential investments, interpret complex technical indicators like RSI and MACD, integrate breaking market news, and analyse intricate corporate financial statements - all simultaneously and continuously. This cognitive burden alone creates a significant barrier to effective decision-making.

Biases in Investment Decisions

Investors routinely exhibit recency bias, giving too much weight to recent market events while ignoring longer-term patterns. Confirmation bias leads them to seek information that validates their existing investment theses while disregarding contradictory evidence.

Risk Perception Challenges

Most retail investors struggle to accurately quantify expected market volatility and fail to align their actual risk tolerance with their portfolio construction. There's a pervasive blindness to tail risk - the probability and potential impact of extreme market events that occur more frequently

than standard models predict. Investors also frequently misunderstand asset correlations, especially how seemingly diverse investments may become highly correlated during market stress, undermining diversification benefits precisely when they're most needed.

Asset Selection & Portfolio Management & Optimization

Individual investors often select assets based on familiarity or recent performance rather than fundamental analysis, leading to suboptimal portfolios from the outset. They frequently struggle with diversification, either over-diversifying and diluting potential returns or under-diversifying and taking on excessive concentration risk. They also tend to focus single-dimensionally on returns without properly accounting for risk. Rebalancing timing becomes a persistent question without clear answers, leading to either excessive trading or neglect of portfolio drift.

Technological Limitations

Individual investors face significant technological disadvantages compared to institutional market participants. The computational constraints are substantial - retail investors lack the processing power to analyse large market datasets, the capabilities to process information in real-time, access to sophisticated automated execution systems, and the ability to rigorously back test strategies against historical data.

DIFFERENTIATION

The product represents a significant advancement over traditional portfolio management tools and even other machine learning-based approaches.

TRADITIONAL PORTFOLIO MANAGEMENT PLATFORMS

1. MetaTrader (MT4/MT5)

Approach: MetaTrader is widely used for online retail foreign exchange trading, focusing primarily on technical analysis and rule-based trading. It operates on fixed, rule-based strategies that remain static once implemented and cannot adjust to changes in market conditions.

Differentiation: The DDPG3 system dynamically adapts to market changes through reinforcement learning, unlike MetaTrader's static approach. While MetaTrader struggles with integrating alternative data sources, the new system incorporates a rich set of technical indicators, risk metrics, and can be expanded to include sentiment and macroeconomic factors. This approach also addresses MetaTrader's inadequate portfolio management capabilities through sophisticated position sizing and risk budgeting.

2. Interactive Brokers (TWS)

Approach: Interactive Brokers' Trader Workstation (TWS) provides professional-grade tools for global financial market trading but requires manual strategy development and back testing. While sophisticated, it lacks built-in machine learning capabilities, making it time-consuming to develop dynamic strategies.

Differentiation: The DDPG3 approach automates the strategy development and optimization process through reinforcement learning, eliminating the manual effort required in TWS. While TWS users must manually adjust strategies as market conditions change, the DDPG3 architecture continuously learns and adapts in real-time. Additionally, the integrated risk management component provides more sophisticated portfolio optimization than TWS's traditional tools.

3. Bloomberg Terminal

Approach: The Bloomberg Terminal provides professionals with real-time financial market data and trading capabilities but at a prohibitively high cost. Despite its comprehensive data, it has limited machine learning integration and struggles with dynamic portfolio management.

Differentiation: The DDPG3 system offers advanced portfolio optimization techniques at a fraction of Bloomberg's cost. While Bloomberg's complex interface can be overwhelming for newcomers, the DDPG3 architecture provides a more transparent and interpretable approach to investment decisions. The system's ability to learn optimal risk management strategies addresses Bloomberg's limitations in dynamic portfolio management.

Machine Learning (ML)-Powered Platforms

4. QuantConnect

Approach: QuantConnect allows users to design, backtest, and deploy machine learning-based strategies with integration for libraries like TensorFlow and scikit-learn. However, it presents a steep learning curve, requires resource-intensive computation for complex strategies, and offers limited built-in ML algorithms.

Differentiation: The DDPG3 system provides ready-to-use reinforcement learning architecture specialized for portfolio management, eliminating the need to build ML models from scratch. The integrated optimization approach achieves superior results with lower computational overhead compared to QuantConnect's general-purpose ML integration. While QuantConnect users must create their own portfolio optimization logic, the DDPG3 system has this capability built into its core architecture.

5. Alpaca

Approach: Alpaca offers a commission-free trading platform with API access for algorithmic trading but lacks native ML support. Users must build their own models from scratch and

integrate them with Alpaca's API, requiring significant custom development. It also has limited risk management tools.

Differentiation: The DDPG3 system provides a pre-built, specialized RL algorithm for both allocation and risk management, eliminating the extensive custom development needed with Alpaca. While Alpaca users must develop their own risk management framework, the DDPG3 component handles this automatically with sophisticated position sizing and risk budgeting. This integrated approach also eliminates the need to connect disparate systems, as would be required when using Alpaca with custom ML models.

6. Quantopian

Approach: Quantopian was a popular algorithmic trading platform that allowed users to create and backtest ML-based strategies using Python. However, it has been discontinued, required integration with external brokers, and lacked focus on portfolio optimization.

Differentiation: The DDPG3 system addresses Quantopian's limitations by providing a comprehensive solution that includes both strategy development and portfolio optimization in one integrated platform. While Quantopian focused primarily on individual trading algorithms, the DDPG3 approach explicitly addresses portfolio-level optimization and risk management. The system also offers more specialized reinforcement learning algorithms tailored to financial markets versus Quantopian's general-purpose ML tools.

TARGET AUDIENCE AND PROJECT SCOPE

The project addresses the significant knowledge and resource gap faced by individuals involved in financial trading, particularly on the Toronto Stock Exchange (TSX).

Market Reach

The scope of this project extends beyond traditional portfolio management tools to offer transformative potential across multiple domains. With the online trading market growing at 15% CAGR, our solution addresses an expanding audience of tech-savvy traders who represent over \$29.2 trillion in investable assets. This unprecedented market opportunity positions our project to become a cornerstone technology in the democratization of advanced portfolio management.

Target Audience

This project addresses both institutional stakeholders and the retail trading market, we mainly focus on retail trader and beginner traders, with a particular focus on Canadian investors who trade on the Toronto Stock Exchange (TSX). Retail traders are at a significant disadvantage compared to institutional investors, with over 75% losing money annually due to ineffective portfolio strategies and inadequate risk management. Most existing solutions for Canadian retail investors are either too generic, offering static pre-defined strategies without

consideration for TSX-specific factors, or too complex for non-professional users. Most existing solutions for Canadian retail investors are either too generic, offering static pre-defined strategies without consideration for TSX-specific factors, or too complex for non-professional users. Our target audience within the Canadian retail sector includes:

- Active individual investors ranging from beginners to intermediate-level traders
- Tech-savvy millennials and Gen Z who prefer modern AI-driven tools
- Part-time traders who need efficient portfolio management solutions with minimal time investment specifically for their TSX-focused portfolios

Financial Impact

With 75% of retail traders currently losing money annually, the potential financial impact is substantial. Even a modest improvement in success rates could translate to billions in preserved capital across the retail segment, with the system's risk management component specifically targeting the reduction of catastrophic losses. This financial protection aspect creates significant societal value beyond pure investment returns, potentially changing the narrative around retail participation in financial markets from wealth destruction to wealth creation.

Technical Application

The technical scope encompasses application across multiple asset classes primarily focused on the Toronto Stock Exchange (TSX). The system has been specifically trained on Canadian securities, making it particularly valuable for investors in the Canadian market. The model can analyse almost entire TSX-listed securities simultaneously, making it scalable to cover entire market segments or indices of the Canadian market, with potential to extend to global markets in future iterations. This specialized focus on the TSX provides Canadian retail investors with a tailored solution that understands the unique characteristics and behaviours of their domestic market, while the architecture remains adaptable for expansion to other markets.

RESEARCH

Approaches, Tools, and Techniques Tried

Initial Framework Selection: FinRL

The project began with the implementation of FinRL, an open-source framework specifically designed for applying deep reinforcement learning (DRL) in quantitative finance. Several compelling reasons motivated this initial choice:

Integration with Yahoo Finance: Yahoo Finance provides comprehensive financial data, including stock quotes, historical prices, and financial metrics. FinRL's built-in integration significantly streamlined the data collection process, particularly for Toronto Stock Exchange data required by this project. This integration promised to eliminate the typically tedious

process of financial data acquisition and preprocessing, allowing for more focus on algorithm development.

Pre-built State-of-the-Art RL Models: FinRL offered a range of pre-implemented reinforcement learning algorithms specifically tuned for financial applications, including Proximal Policy Optimization (PPO), Advantage Actor-Critic (A2C), and Soft Actor-Critic (SAC). These implementations provided a valuable starting point, potentially reducing development time by eliminating the need to build these algorithms from scratch.

Integrated Financial Indicators: One of FinRL's most attractive features was its capability to automatically calculate and incorporate technical indicators into the dataset without requiring explicit implementation. These indicators—including moving averages, relative strength index (RSI), MACD, and Bollinger Bands—are crucial for providing context and additional information about stock behavior patterns, potentially enhancing model performance.

Limitations and Challenges Encountered

Despite its promising features, several significant challenges emerged during the implementation of FinRL that ultimately led to its rejection as the primary framework:

Outdated Dependencies and Limited Maintenance: The framework relied on outdated libraries and lacked regular maintenance updates. This created compatibility issues with current Python environments and other required packages, resulting in frequent runtime errors and implementation difficulties.

Data Processing Limitations: When attempting to use FinRL's feature engineering capabilities, we encountered critical issues where the framework would eliminate the majority of tickers from our dataset during the technical indicator calculation process. This substantial data loss significantly compromised our ability to analyze a diverse set of stocks, which was essential for effective portfolio construction.

Insufficient Documentation and Community Support: As a relatively new and specialized framework, FinRL lacked comprehensive documentation and an active support community. This absence became particularly problematic when troubleshooting implementation issues, as solutions and workarounds were not readily available online. The limited resources for bug resolution and optimization made development progress increasingly difficult.

Approaches, Tools, and Techniques Approved

After the rejection of FinRL as a comprehensive framework, the team approved an alternative approach for extracting Toronto Stock Exchange data while developing custom reinforcement learning models for portfolio management. This selective approach was chosen for several compelling reasons:

Custom Implementation of RL Algorithms:

The team shortlisted three reinforcement learning algorithms for custom implementation: Proximal Policy Optimization (PPO), Advantage Actor-Critic (A2C), and Soft Actor-Critic (SAC). By building these algorithms independently of the FinRL framework, the team gained complete control over their architecture, hyperparameters, and optimization procedures, enabling fine-tuning specifically for financial applications.

The team conducted a comprehensive evaluation of four reinforcement learning algorithms: Deep Deterministic Policy Gradient (DDPG), Proximal Policy Optimization (PPO), Soft Actor-Critic (SAC), and Advantage Actor-Critic (A2C). After rigorous testing, DDPG3 was chosen for the portfolio allocation and risk management.

PPO Implementation for Portfolio Allocation

The Proximal Policy Optimization algorithm was implemented through a custom Gymnasium environment modeling the investment setting. The state space incorporated 30-day historical lookback windows of key technical indicators, including Moving Averages (Volume and Price), Relative Strength Index (RSI), Moving Average Convergence Divergence (MACD), Bollinger Bands, Momentum, Volatility, and Returns. The continuous action space enabled allocation weights across stocks, with these technical indicators providing a comprehensive view of market dynamics. The reward function was based on log returns or portfolio-level returns depending on configuration. The team developed an actor-critic network with configurable architecture and implemented it using Stable-Baselines3 for compatibility and robustness. This implementation enabled the system to effectively learn asset selection and dynamic allocation based on market trends, momentum, and historical patterns, leveraging the rich feature space created by the carefully selected technical indicators.

A2C Implementation for Risk Management

The Advantage Actor-Critic algorithm was implemented using a custom Gymnasium environment (PortfolioRiskEnv) for portfolio risk simulation. The state space included stock features, risk metrics, and portfolio status, while the action space enabled continuous control of position sizing and risk budget allocation. A multifaceted reward function balanced returns against volatility, drawdown, and risk efficiency. The team developed an optimized neural network architecture with 128-unit layers and implemented N-step returns for improved temporal credit assignment. This approach allowed the system to manage risk dynamically and adapt to changing market conditions.

SAC Implementation for Risk Management

The Soft Actor-Critic (SAC) algorithm was implemented through Gymnasium environment (PortfolioRiskEnv) designed to simulate comprehensive portfolio risk management. It also used the StableBaseline3 for the model building. The model integrates key technical indicators including Relative Strength Index (RSI), Moving Average Convergence Divergence (MACD), Bollinger Bands, and trend strength metrics to capture multidimensional market dynamics, enabling adaptive risk assessment and intelligent investment decision-making. The SAC algorithm tries balances exploration and exploitation, creating a robust framework for managing portfolio risk that can dynamically respond to changing market conditions. The technical indicators provide a comprehensive feature space encompassing momentum, volatility, and trend characteristics, transforming raw financial data into actionable insights that support nuanced risk-return trade-offs and sophisticated portfolio allocation strategies.

DDPG3 Implementation for Portfolio Allocation and Risk Management

The Deep Deterministic Policy Gradient (DDPG3) algorithm was implemented through a sophisticated custom Gymnasium environment designed to comprehensively model portfolio investment dynamics. The state space integrated a carefully curated set of technical indicators, including Returns, Volume Moving Average, Price Moving Average, Relative Strength Index (RSI), Moving Average Convergence Divergence (MACD), Volatility, and Momentum. These indicators provided a multidimensional view of market behavior, enabling the algorithm to capture complex market dynamics and make intelligent investment decisions. The continuous action space allowed for precise allocation weights and risk management parameters, while the actor-critic network architecture learned intricate relationships between market indicators and optimal investment strategies. By balancing portfolio returns against comprehensive risk metrics, the DDPG3 implementation could develop adaptive strategies for asset selection, dynamically allocate resources, and manage risk exposure across varying market conditions, effectively transforming raw financial data into sophisticated portfolio management decisions.

SYSTEM ARCHITECTURE

AI/ML Model Usage

Deep Deterministic Policy Gradient (DDPG) Model

- **Architecture:** Actor-critic network with MLP-based policy and value networks
- **Implementation:** Built using Stable-Baselines3 library with custom modifications
- **Training Data:** Historical stock data spanning approximately 20 years with technical indicators
- **Hyperparameters:**
 - Learning rate: $1e-3$
 - Buffer size: 1,000,000
 - Batch size: 128
 - Tau (soft update factor): 0.005
 - Gamma (discount factor): 0.99
 - Action noise: Normal with $\sigma=0.1$
- **Policy Network:** Maps state observations to portfolio allocation weights
- **Value Network:** Evaluates expected returns from state-action pairs
- **Training Process:** 500,000 timesteps in 5 stages of 100,000 each

- **Output:** Continuous action values representing allocation percentages for stocks

Soft Actor-Critic (SAC) Model Experimentation

- **Purpose:** Experimental approach for specialized risk management aspects
- **Architecture:** Entropy-regularized actor-critic algorithm with twin Q-networks
- **Implementation:** Built using Stable-Baselines3 SAC implementation
- **Policy Structure:**
 - Two hidden layers with 128 neurons each for both policy and Q-functions
 - ReLU activation functions
 - Automatic entropy coefficient tuning
- **Hyperparameters:**
 - Learning rate: $3e-4$
 - Buffer size: 1,000
 - Batch size: 64
 - Tau (soft update): 0.005
 - Gamma (discount factor): 0.99
 - Training frequency: 1 step
- **Training Process:**
 - Total of 10,000 timesteps divided into 5 chunks for stability
 - Checkpoint saving every 5,000 steps
 - Regular evaluation during training
- **Feature Selection:** Dynamic feature selection from available data columns including:
 - Required features: daily_return, close, volume
 - Optional features: volatility_20d, drawdown, volume_change, momentum metrics, technical indicators
- **Environment:** Custom PortfolioRiskEnv designed for risk-focused portfolio management
- **Results:** While showing promise in specialized risk contexts, ultimately the integrated DDPG approach was selected as the primary model for the unified portfolio allocation and risk management system

Data Model

1. Stock Data Schema

- **Primary Dataset:** Time-series stock data with the following structure:

```
{
  "date": Datetime,
  "tic": String,      // Ticker symbol
  "open": Float,      // Opening price
  "high": Float,      // High price
  "low": Float,       // Low price
  "close": Float,     // Closing price
  "volume": Integer,  // Trading volume
  "day": Integer,     // Day index
  "industry": String, // Industry classification
  "sector": String,   // Sector classification
}
```

- **Technical Indicators:** Added during preprocessing

```
{
  "rsi": Float,      // Relative Strength Index
  "macd": Float,     // MACD value
  "cci": Float,      // Commodity Channel Index
  "adx": Float,      // Average Directional Index
  "bb_upperband": Float, // Bollinger Band upper
  "bb_lowerband": Float, // Bollinger Band lower
  "mom": Float,      // Momentum
}
```

- **Risk Metrics:** Calculated for portfolio management

```
{
  "volatility": Float, // Historical volatility
  "beta": Float,      // Market correlation
  "sharpe": Float,    // Sharpe ratio
  "max_drawdown": Float, // Maximum drawdown
  "risk_category": String // Low/Medium/High risk
}
```

Data Storage

- **Primary Storage:** CSV files for historical stock data (no database requirement)
- **In-Memory Processing:** Data loaded and processed in memory using pandas DataFrames
- **Serialization:** Trained models saved using pickle/joblib format

- **Caching Strategy:** StockPortfolioIterator implements memory-efficient batching for large datasets

Data Flow

1. Data Gathering Pipeline

1. Source Acquisition:

- Historical stock data retrieved from Yahoo Finance
- Data includes OHLCV (Open, High, Low, Close, Volume) information
- Additional metadata (sector, industry) collected from financial APIs

2. Initial Processing:

- Data loaded into pandas DataFrames
- Date columns converted to datetime format
- Missing values identified and flagged

2. Data Cleaning Pipeline

1. Missing Data Handling:

- For low-frequency data: Rows with NaN values deleted
- For high-frequency data: Forward filling with last valid close price
- Volume NaN values filled with 0

2. Outlier Detection and Handling:

- Interquartile Range (IQR) method applied
- Extreme values clipped to reduce impact on model training
- Standardization using StandardScaler to normalize features

3. Feature Engineering Pipeline

1. Technical Indicator Generation:

- Moving averages (5, 10, 20, 60-day windows)
- Momentum indicators (RSI, MACD)
- Volatility indicators (Bollinger Bands)
- Custom indicators (return rates, volatility measures)

2. Risk Metric Calculation:

- Rolling volatility (standard deviation of returns)

- Drawdown measurements
- Sharpe ratio calculation
- Beta calculation relative to market index

3. Feature Scaling:

- Standardization to zero mean and unit variance
- Batch-wise processing for memory efficiency
- Outlier clipping using IQR method

4. Training Data Preparation

1. Data Organization:

- Grouping by date for cross-sectional analysis
- Creating lookback windows (30 days) for temporal features
- Batch processing using StockPortfolioIterator

2. Dataset Splitting:

- Chronological splitting for time-series data
- Training on earlier data periods
- Testing/validation on most recent data periods

3. Integration Points

- **Model Interface:** Standardized predict() method for model interaction
- **Data Interface:** Consistent DataFrame structure for data exchange between components
- **Environment Interface:** Standard Gymnasium interface (reset, step, render) for RL training

4. Deployment Considerations

- **Containerization:** Models packaged using Docker for consistent execution
- **Model Serving:** Simple API endpoints for portfolio recommendations
- **Scalability:** Batch processing design allows handling larger stock universes
- **Resource Management:** Memory-efficient iterators for processing large datasets

Features Implemented in the Project

Budget-Based Investment Planning

A standout feature of the project is the Budget-Based Investment Planning system, which allows users to input the total amount they are willing to invest. Once the budget is set, the model uses financial analysis and market data to determine the ideal percentage of the capital that should be actively invested in the stock market versus what should be kept as reserve or liquid. It then generates a personalized investment plan that includes specific stock recommendations, detailing how much to invest in each stock based on risk level, diversification principles, and projected performance. This removes the burden of manual research and guesswork for the user. Additionally, the system enhances investment timing by identifying the best date to invest, taking into account market trends or volatility cycles. To complete the strategy, it also suggests an optimal withdrawal date, allowing users to plan their exit and lock in potential gains or limit exposure to downturns. This end-to-end automation of investment strategy—from budget input to withdrawal planning—not only simplifies the process for users but also instills confidence, especially for those who may not have deep financial expertise. It ensures every rupee or dollar is put to work effectively, helping users achieve their investment goals with strategic, time-sensitive guidance.

Risk Level Selection (Low, Medium, High)

One of the key features of the stock market portfolio allocation project is the Risk Level Selection, which allows users to choose their preferred investment risk profile—Low, Medium, or High. This customization ensures that the portfolio suggestions are aligned with each user's financial goals and comfort with market volatility. Users opting for a low-risk level are recommended stable, less volatile assets such as blue-chip stocks or ETFs, making it ideal for conservative investors or beginners. Those selecting medium risk receive a balanced allocation that mixes growth and stability, typically including a blend of mid-cap and large-cap stocks. For users choosing high-risk, the system suggests high-growth potential assets, such as small-cap or sector-specific stocks that carry higher returns but also greater volatility. This feature empowers users to invest responsibly by understanding the trade-off between risk and reward, while also promoting better decision-making and financial planning. Whether a user is a cautious beginner or an experienced investor, this risk-based approach helps build a well-structured and personalized portfolio, reducing stress and enhancing long-term investment confidence.

Potential Future Features that can be implemented

Real-Time Market Data Integration

To enhance the accuracy and responsiveness of the portfolio allocation model, integrating real-time market data is a crucial next step. This involves connecting the model to a financial data provider, such as Alpha Vantage, IEX Cloud, or Yahoo Finance, which offers APIs to access

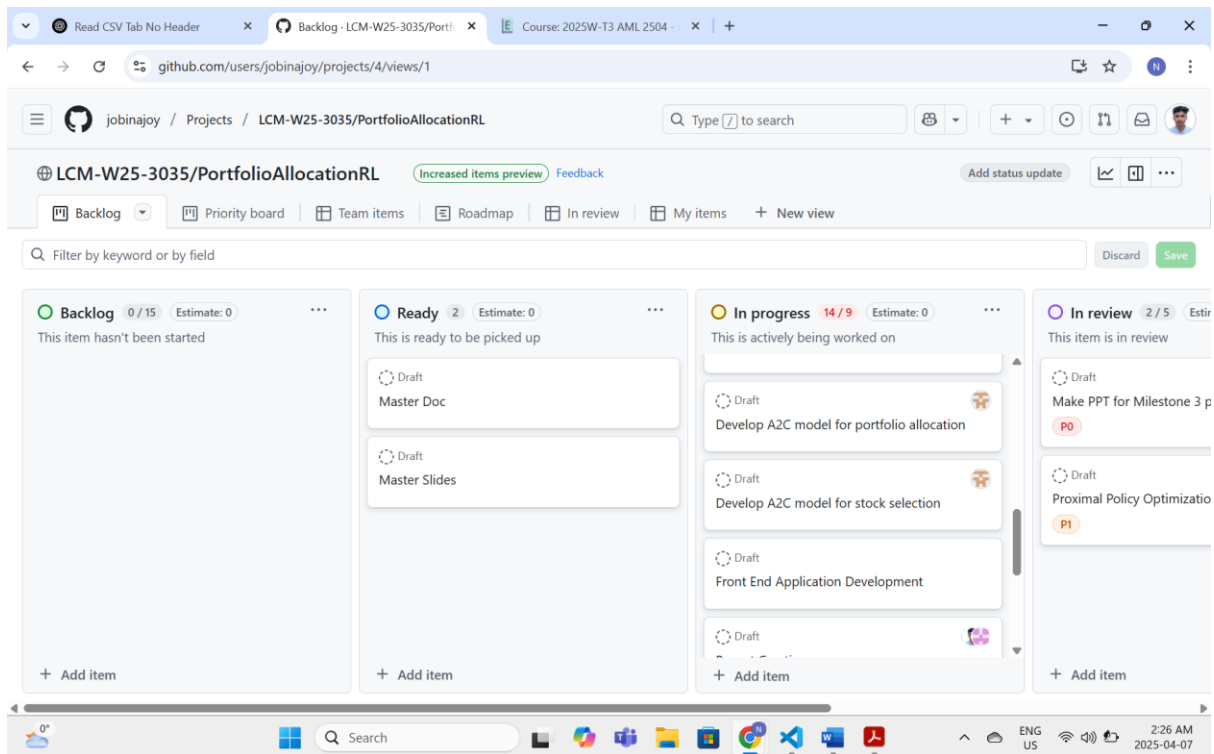
up-to-date stock prices, historical data, and financial metrics. Once integrated, the system can dynamically fetch the latest stock prices and use them to calculate more accurate investment allocations, track portfolio performance, and respond to market changes in real time. For example, instead of relying on static or outdated prices, the model can update the suggested stock quantities based on the user's budget and current market rates, ensuring the investment recommendations are precise and actionable. In addition to using real-time data for live inference, where the model applies its learned policy to current market conditions, it is also important to implement periodic retraining of the model. This ensures the system continues to adapt to evolving market behavior and does not rely solely on older patterns that may no longer be relevant. This integration also enables features like live portfolio valuation, timely buy/sell suggestions, and instant alerts when a stock hits a user-defined price target. Overall, combining real-time data with periodic retraining adds a significant layer of intelligence, adaptability, and credibility to the platform, making it more valuable for users seeking timely and data-driven investment decisions.

User Portfolio Tracking & Analytics Dashboard

The User Portfolio Tracking & Analytics Dashboard is a powerful feature designed to give users a clear, personalized view of their investment journey. This dashboard enables users to track the growth of their investments over time, offering real-time insights into how their portfolio is performing. It displays detailed metrics such as historical performance trends, current asset allocation, and individual stock performance, helping users understand where their capital is distributed and how each asset is contributing to overall returns. Additionally, the dashboard includes a risk score indicator, allowing users to assess whether their portfolio aligns with their chosen risk level (low, medium, or high). One of the most useful elements of the dashboard is its ability to send alerts or notifications when a particular stock underperforms, helping users make timely decisions to rebalance or exit positions. Overall, this feature enhances user engagement, supports informed decision-making, and provides transparency and control—all in an intuitive, easy-to-understand visual format tailored to each investor's goals and strategy.

Project Board Setup and Management

GitHub : <https://github.com/users/jobinajoy/projects/4/views/1>



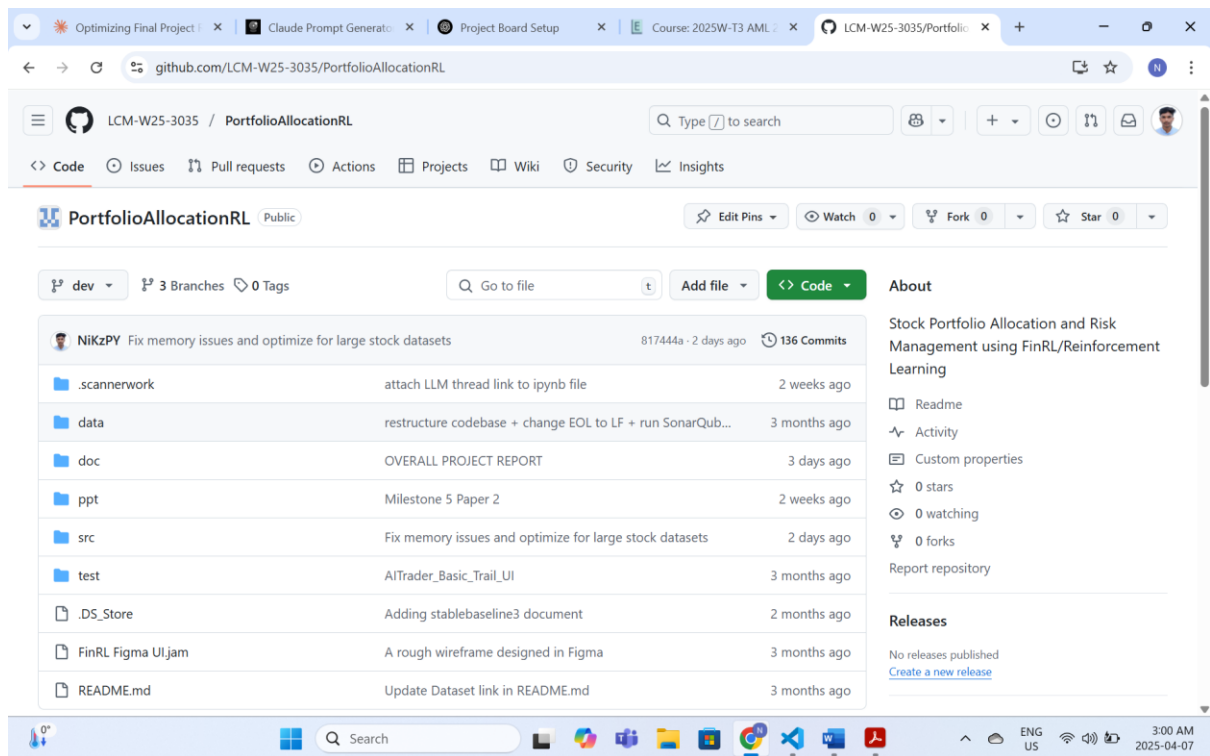
Board Structure

1. **Backlog:** All identified tasks awaiting prioritization
2. **To Do:** Prioritized tasks ready for implementation
3. **In Progress:** Tasks currently being worked on
4. **Review:** Tasks awaiting peer review
5. **Testing:** Features under testing
6. **Done:** Completed and verified tasks

Task Management Process

- **Task Creation:** Each feature or bug fix is documented as an issue with detailed requirements
- **Assignment:** Tasks are assigned to specific team members based on expertise and workload
- **Labels:** Issues are categorized using labels such as "enhancement," "bug," "documentation," or "refactor"
- **Prioritization:** Tasks are ranked by importance and dependencies
- **Tracking:** Progress updates are provided through issue comments

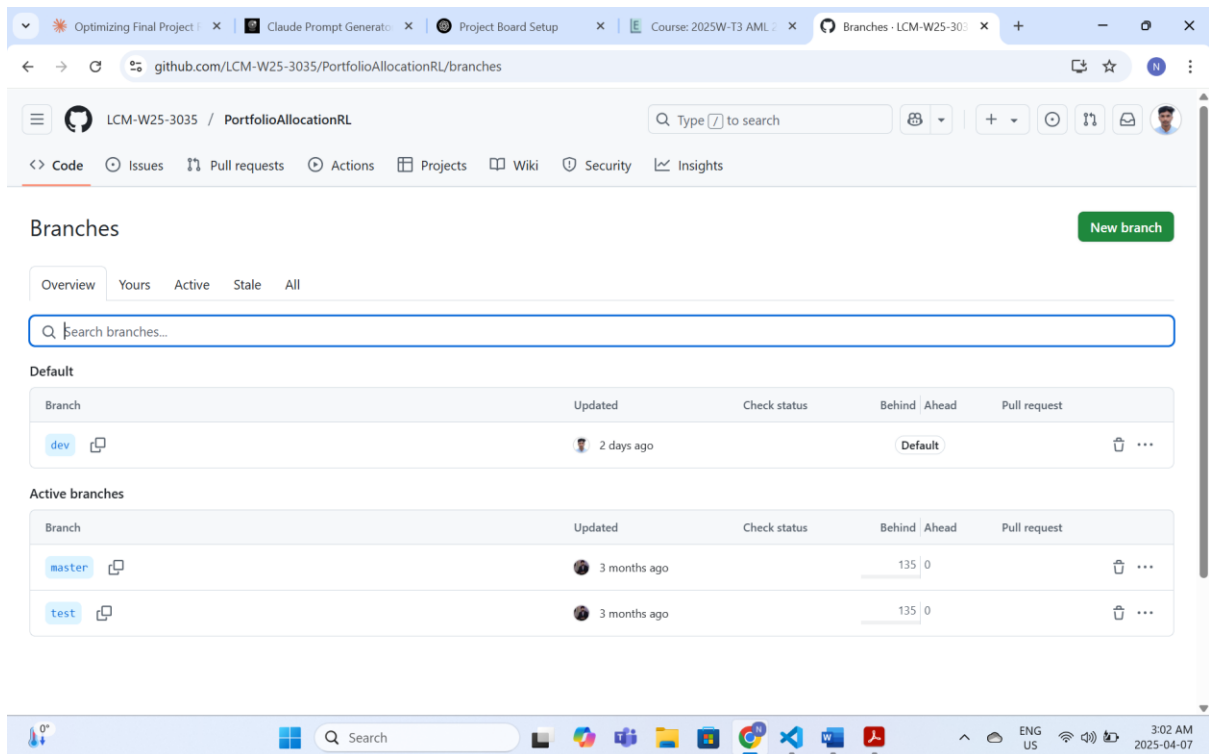
Repository Architecture



The repository contains several top-level directories and files:

- `.scannerwork`: This directory typically holds configuration and output files for code analysis tools.
- `data`: This folder is intended for datasets used in the project.
- `doc`: Contains documentation related to the project.
- `ppt`: This directory store presentation files associated with the project.
- `src`: The source code for the project is housed here.
- `test`: This folder contains test scripts and related resources.
- `.DS_Store`: A system file created by macOS to store custom attributes of a folder.
- `FinRL Figma UI.jam`: This file related to user interface designs or collaborative planning.
- `README.md`: The main readme file providing an overview of the project.

Branches and Their Functions:



Main Branches

1. **master**: The production-ready branch containing stable code
 - Contains fully tested and reviewed code
 - Represents the main deployment version
2. **test**: Testing and quality assurance branch
 - Used for integration testing before merging to master
 - Contains features that have passed initial development but need verification
3. **dev**: Active development branch
 - Where new features are being developed and integrated
 - Source for feature-specific work before moving to test

Files/Folders and Their Contents:

- **.scannerwork**: Likely contains configuration and output files for code analysis tools.
- **data**: Intended for datasets used in the project. The README mentions that the portfolio datasets include stocks from various companies to ensure diversification.
- **doc**: Houses documentation files, including the master project report.
- **ppt**: Contains presentation materials related to the project.

- **src:** The main source code for the project is likely located here.
- **test:** Includes test scripts to validate the project's functionality.
- **.DS_Store:** A macOS system file that can typically be ignored.
- **FinRL Figma UI.jam:** Potentially related to UI/UX design or collaborative planning.
- **README.md:** Provides an overview of the project, including links to the job board, master document, data sources, and dataset.

Git Commits of Team Members

Commits

dev abhi-shek-3917 All time

Commits on Apr 7, 2025

Master Documentation Final
abhi-shek-3917 committed 37 minutes ago f88b171

Commits on Mar 31, 2025

add prompt
abhi-shek-3917 committed last week 19f6cef

Merge branch 'dev' of <https://github.com/LCM-W25-3035/PortfolioAllocationRL> into dev
abhi-shek-3917 committed last week f47bbc4

worked on fixing ranking error in a2c model
abhi-shek-3917 committed last week 18834a2

Commits on Mar 24, 2025

updated a2c model
abhi-shek-3917 committed 2 weeks ago 38ab6b8

Commits on Mar 17, 2025

Merge pull request #28 from LCM-W25-3035/dev_abhi Verified 7f9cb1b

a2c model tuning
abhi-shek-3917 committed 3 weeks ago 9cbb18b

Commits on Mar 10, 2025

Merge pull request #25 from LCM-W25-3035/dev_abhi Verified c3f2867

A2c model implementation
abhi-shek-3917 committed last month ecbcc5d

Commits on Feb 17, 2025

DRL model using A2C model
abhi-shek-3917 authored on Feb 17 Verified 7b726b8

Commits on Feb 3, 2025

Merge pull request #14 from LCM-W25-3035/dev_abhishek Verified c3481a5

Understanding StockPortfolioEnv
abhi-shek-3917 committed on Feb 3 9c954ca

Commits on Jan 21, 2025

adding the flowchart to the project proposal
abhi-shek-3917 committed on Jan 21 7eebe4a

Commits on Jan 17, 2025

Merge pull request #4 from LCM-W25-3035/dev_abhishek Verified 4e2b38d

Traditional portfolio allocation versus Reinforcement Learning-based methods
abhi-shek-3917 committed on Jan 17 7d72473

Commits

7f9cb1b
c3481a5
c3f2867
7b726b8
9c954ca
7eebe4a
4e2b38d
7d72473

Master Documentation Final
add prompt
Merge branch 'dev' of <https://github.com/LCM-W25-3035/PortfolioAllocationRL> into dev
worked on fixing ranking error in a2c model
updated a2c model
Merge pull request #28 from LCM-W25-3035/dev_abhi
a2c model tuning
Merge pull request #25 from LCM-W25-3035/dev_abhi
A2c model implementation
DRL model using A2C model
Merge pull request #14 from LCM-W25-3035/dev_abhishek
Understanding StockPortfolioEnv
adding the flowchart to the project proposal
Merge pull request #4 from LCM-W25-3035/dev_abhishek
Traditional portfolio allocation versus Reinforcement Learning-based methods

Sn

Screen

Commits

	jobinajoy	All time
Commits on Apr 7, 2025		
Update README.md	Verified 96b1f88	<>
jobinajoy authored 2 hours ago		
Merge pull request #35 from LCM-W25-3035/dev_job	Verified 5ca3f41	<>
jobinajoy authored 2 hours ago		
Master Slide Project	d9b3d64	<>
jobinajoy committed 2 hours ago		
Merge pull request #34 from LCM-W25-3035/dev_job	Verified cd8c177	<>
jobinajoy authored 3 hours ago		
Code for portfolio recommendation and risk management.	4e5b65c	<>
jobinajoy committed 3 hours ago		
Commits on Mar 31, 2025		
Merge pull request #33 from LCM-W25-3035/dev_job	Verified 8e9573f	<>
jobinajoy authored last week		
Hyper parameter tuning	085c152	<>
jobinajoy committed last week		
Commits on Mar 24, 2025		
Add files via upload	Verified 894c567	<>
jobinajoy authored 2 weeks ago		
Commits on Mar 17, 2025		
Merge pull request #29 from LCM-W25-3035/dev_job	Verified 79f9100	<>
jobinajoy authored 3 weeks ago		
ddpg model without Finrl Stock Portfolio Data Iterator for Deep Reinforcement Learning .main code that calls the function and training the model	9c2e38c	<>
jobinajoy committed 3 weeks ago		
Commits on Mar 10, 2025		
Merge pull request #26 from LCM-W25-3035/dev_job	Verified d4cc0d0	<>
jobinajoy authored last month		
Added the code for back test	173e46e	<>
jobinajoy committed last month		
Commits on Feb 17, 2025		
Merge pull request #19 from LCM-W25-3035/dev_job	Verified b88be75	<>
jobinajoy authored on Feb 17		
Model building DDPG	7a2f82f	<>
jobinajoy committed on Feb 17		
Commits on Feb 13, 2025		
Merge pull request #17 from LCM-W25-3035/dev_job	Verified fd05cc6	<>
jobinajoy authored on Feb 13		
Added the code for data cleaning - data preprocessing	011c257	<>
jobinajoy committed on Feb 13		
Commits on Feb 3, 2025		
Merge pull request #12 from LCM-W25-3035/dev_job	Verified 48f999b	<>
jobinajoy authored on Feb 3		
Analysis -- Understanding the relation	e123b45	<>
jobinajoy committed on Feb 3		
Commits on Jan 27, 2025		
Merge pull request #9 from LCM-W25-3035/dev_job	Verified 81cb584	<>
jobinajoy authored on Jan 27		
Added the code for visulaization for Analysis + Insight drawn from the Analysis.	acd0aab9	<>
jobinajoy committed on Jan 27		

Commits on Jan 21, 2025	
Merge pull request #7 from LCM-W25-3035/dev_job	Verified 0f65975
jobinajoy authored on Jan 21	
Added analysis visulaization	9Ffc89a
jobinajoy committed on Jan 21	
Merge pull request #6 from LCM-W25-3035/dev_job	Verified 3fc9be3
jobinajoy authored on Jan 21	
Data exploration+MasterSlide+ Analysis	ceadb47
jobinajoy committed on Jan 21	
Commits on Jan 20, 2025	
Update README.md	Verified 679d9ff
jobinajoy authored on Jan 20	
Commits on Jan 16, 2025	
Merge pull request #1 from LCM-W25-3035/dev_job	Verified b535de8
jobinajoy authored on Jan 16	
Format of the final master doc and project proposal document	7412cc3
jobinajoy committed on Jan 16	
Commits on Jan 15, 2025	
Update README.md	Verified 84390f1
jobinajoy authored on Jan 15	

Commits

dev	tarunsamuel7	All time
Commits on Apr 7, 2025		
add init file for UI data folder	38bc679	
tarunsamuel7 committed 41 minutes ago		
add llm thread link to streamlit app	9bfF2d3	
tarunsamuel7 committed 2 hours ago		
add llm thread link	8b5f2d7	
tarunsamuel7 committed 2 hours ago		
add Streamlit UI with HuggingFace dataset integrated	98d5d4d	
tarunsamuel7 committed 3 hours ago		
implement PPO with custom budget input	e1da0be	
tarunsamuel7 committed 5 hours ago		
Commits on Mar 31, 2025		
Merge branch 'dev' of https://github.com/LCM-W25-3035/PortfolioAllocationRL into dev	db6ce2b	
tarunsamuel7 committed last week		
fix medium risk categorization error in A2C stock ranking v1	60be8ea	
tarunsamuel7 committed last week		
Commits on Mar 24, 2025		
Merge branch 'dev' of https://github.com/LCM-W25-3035/PortfolioAllocationRL into dev	75862cd	
tarunsamuel7 committed 2 weeks ago		
attach LLM thread link to ipynb file	e828d7c	
tarunsamuel7 committed 2 weeks ago		
rank stocks based on risk using A2C	978720b	
tarunsamuel7 committed 2 weeks ago		

Commits on Mar 18, 2025	develop a2c for stock selection -without FinRL tarunsamuel7 committed 3 weeks ago	b8a3091		<>
Commits on Mar 10, 2025	implement model to select stocks using A2C tarunsamuel7 committed last month	e8bd8c4		<>
Commits on Feb 17, 2025	implement A2C DRL agent tarunsamuel7 committed on Feb 17	88e234c		<>
Commits on Feb 8, 2025	Merge pull request #15 from LCM-W25-3035/create-pr-tutorial tarunsamuel7 authored on Feb 8	3880ee1	Verified	<>
	some commit tarunsamuel7 committed on Feb 8	f478994		<>
Commits on Feb 3, 2025	execute and understand portfolio allocation using FinRL tarunsamuel7 committed on Feb 3	c8e064a		<>
Commits on Jan 20, 2025	add problem definition doc tarunsamuel7 committed on Jan 20	832146a		<>
	fetch ticker info and merge with chart data tarunsamuel7 committed on Jan 20	d3e0da6		<>
Commits on Jan 19, 2025	Merge branch 'dev' of https://github.com/LCM-W25-3035/PortfolioAllocationRL into dev tarunsamuel7 committed on Jan 19	2f48889		<>
	create documentation for SonarQube local server installation tarunsamuel7 committed on Jan 19	4aadd6b		<>
Commits on Jan 16, 2025	Merge branch 'dev' of https://github.com/LCM-W25-3035/PortfolioAllocationRL into dev tarunsamuel7 committed on Jan 16	46e814d		<>
	restructure codebase + change EOL to LF + run SonarQube scanner tarunsamuel7 committed on Jan 16	9b09453		<>

Commits

dev

gamerlion108

All time

Commits on Apr 7, 2025

Add files via upload

gamerlion108

authored 7 minutes ago

Verified

d7f12b9

<>

Merge pull request #37 from LCM-W25-3035/dev_anand

gamerlion108

authored 1 hour ago

Verified

576b49b

<>

Testing final ddpq model

gamerlion108

committed 1 hour ago

094cc6c

<>

Commits on Mar 31, 2025

Merge pull request #32 from LCM-W25-3035/dev_anand

gamerlion108

authored last week

Verified

e188a2e

<>

Have done the backtesting and tested for mutiple recommendation mode.

gamerlion108

committed last week

ef8df8a

<>

Commits on Mar 24, 2025

Milestone 5 Paper 2

gamerlion108

authored 2 weeks ago

Verified

63f1afd

<>

Commits on Mar 18, 2025

Merge pull request #30 from LCM-W25-3035/dev_anand

gamerlion108

authored 3 weeks ago

Verified

f3e6957

<>

Since my code got removed I am doing the checkin again

gamerlion108

committed 3 weeks ago

939da46

<>

Commits on Mar 17, 2025

Code for model evaluation and configure the gynasium - without using Finrl

gamerlion108

committed 3 weeks ago

975b7f5

<>

Commits on Mar 10, 2025	Merge pull request #27 from LCM-W25-3035/dev_anand  06e2c3c  <>  gamerlion108 authored last month Integrating all the tickers into the DDPG model  gamerlion108 committed last month 7eaa229  <>
Commits on Feb 17, 2025	Merge pull request #22 from LCM-W25-3035/dev_anand  cc28c84  <>  gamerlion108 authored on Feb 17 Model Tuning  gamerlion108 committed on Feb 17 85e293f  <>
Commits on Feb 11, 2025	Merge pull request #16 from LCM-W25-3035/dev_anand  82be579  <>  gamerlion108 authored on Feb 11 Combining documents  gamerlion108 committed on Feb 11 d4f2997  <>
Commits on Feb 3, 2025	Merge pull request #13 from LCM-W25-3035/dev_anand  68c884d  <>  gamerlion108 authored on Feb 3 Understanding finrl.plot - get_daily_return, get_baseline, convert_daily_return_to_pyfolio_ts  gamerlion108 committed on Feb 3 44bfb04  <>
Commits on Jan 27, 2025	Merge pull request #10 from LCM-W25-3035/dev_anand  9ff1384  <>  gamerlion108 authored on Jan 27 Visualization code  gamerlion108 committed on Jan 27 35e7ff8  <> Added insights and visualization  gamerlion108 committed on Jan 27 cd1188b  <>
Commits on Jan 20, 2025	Merge pull request #5 from LCM-W25-3035/dev_anand  41a383a  <>  gamerlion108 authored on Jan 20 Big Data Pipeline + Analysis of data  gamerlion108 committed on Jan 20 7878564  <>

dev

NiKzPYAll time

Commits on Apr 7, 2025

Adding System Architecture, Repository Architecture, Code Structure Explanation, Code Quality/SonarQube.

 NiKzPY committed 1 hour ago

9622e81

Overall Project Report Updation

 NiKzPY committed 1 hour ago

7e16778

Merge pull request #36 from LCM-W25-3035/dev_nik

 NiKzPY authored 1 hour ago

2ea35c1

Verified

model taring with all stocks and adding hyperparameter tuning and backtest

 NiKzPY committed 1 hour ago

759c885

Commits on Apr 5, 2025

Fix memory issues and optimize for large stock datasets

 NiKzPY committed 3 days ago

817444a

Commits on Mar 31, 2025

SAC model -fixing error bug

 NiKzPY committed last week

ff42c15

Commits on Mar 24, 2025

Merge pull request #31 from LCM-W25-3035/dev_nikhil

 NiKzPY authored 2 weeks ago

8b1378f

Verified

Adding code for SAC model creation without using FinRL

 NiKzPY committed 2 weeks ago

fac5fbf

Commits on Mar 10, 2025

Merge pull request #24 from LCM-W25-3035/dev_nikhil

 NiKzPY authored last month

ed8d7f8

Verified

Updating the Indicators and trying to optimize hyperparameter tuning

 NiKzPY committed last month

9374c37

Commits on Mar 3, 2025

Merge pull request #23 from LCM-W25-3035/dev_nikhil

NIKzPY authored on Mar 3

Verified

3b28748



Updated code for SAC model creation and adding SAC model tuning

NIKzPY committed on Mar 3

cee948e



Commits on Feb 17, 2025

Merge pull request #21 from LCM-W25-3035/dev_nikhil

NIKzPY authored on Feb 17

Verified

6db83ef



Further implementaion of SAC

NIKzPY authored on Feb 17

Verified

d868a4d



Commits on Feb 3, 2025

uploading the data preprocessing file part 1

NIKzPY authored on Feb 3

Verified

e29b8d9



Commits on Jan 17, 2025

Merge pull request #3 from LCM-W25-3035/dev_nikhil

NIKzPY authored on Jan 17

Verified

6b64ad1



Importance of risk management and Role of technologies like reinforcement learning in modern finance.

NIKzPY committed on Jan 17

5d7de63



Commits on Jan 16, 2025

Merge pull request #2 from LCM-W25-3035/dev_nikhil

NIKzPY authored on Jan 16

Verified

dF9f234



Explains how the industry works + Key processes in portfolio management - doc update

NIKzPY committed on Jan 16

a35e156



Commits on Apr 7, 2025

Overall Project Report

riyoseb8 committed 2 hours ago

686f4a4



Commits on Apr 4, 2025

OVERALL PROJECT REPORT

riyoseb8 committed 4 days ago

597ead6



Commits on Apr 3, 2025

SAC model for risk management

riyoseb8 committed 5 days ago

d918586



Commits on Mar 31, 2025

Merge branch 'dev' of <https://github.com/LCM-W25-3035/PortfolioAllocationRL> into dev

riyoseb8 committed last week

846a04a



Adding SAC model for Risk Management

riyoseb8 committed last week

b99971f



Commits on Mar 25, 2025

Building the SAC Model

riyoseb8 committed 2 weeks ago

4abe4a1



Commits on Mar 10, 2025

Updating the SAC model - Defining the risk indicator and fixing the FeatureEngineer call

riyoseb8 committed last month

352d968



Updating te SAC model

riyoseb8 committed last month

46a8e03



UPDATING DRL_SAC_IMPLEMENTATION - Adding Risk Indicators and Fixing FeatureEngineer Call Error

riyoseb8 committed last month

f9a30f2



Commits on Feb 17, 2025

Merge pull request #20 from LCM-W25-3035/dev_riyo

riyoseb8 authored on Feb 17

Verified

4043a94



Implementating SAC model for portfolio management

riyoseb8 authored on Feb 17

Verified

1f353d8



Commits on Apr 7, 2025

Final PPT

JeffinKR authored 11 minutes ago

Verified

c88eaF6



Final Report - Adding Features Implemented, Deployment Instructions and individual commits

JeffinKR authored 15 minutes ago

Verified

28c7359



Commits on Mar 31, 2025

The error encountered last week has been fixed.

JeffinKR authored last week

Verified

b6b9fa9



Commits on Mar 24, 2025

ddpg_model_without_using_finrl faced some error

JeffinKR authored 2 weeks ago

Verified

c225b32



Commits on Mar 18, 2025

ddpg_model_without_using_finrl_jeffin

JeffinKR authored 3 weeks ago

Verified

e29140e



Commits on Mar 10, 2025

Implement DDPG model

JeffinKR authored last month

Verified

04048cf



Commits on Feb 17, 2025

Merge pull request #18 from LCM-W25-3035/JeffinKR-patch-1

JeffinKR authored on Feb 17

Verified

848bc1e



Significance of Technical Indicators

JeffinKR authored on Feb 17

Verified

e2abc88



Commits on Feb 3, 2025

FinRL Backtest_Plot And Backtest_Stats Analysis

JeffinKR authored on Feb 3

Verified

2ad8233



Commits on Jan 27, 2025

AITrader_Basic_Trail_UI

JeffinKR authored on Jan 27

Verified

42830ec



Commits on Jan 20, 2025

A rough wireframe designed in Figma

JeffinKR authored on Jan 20

Verified

408dbaf



Commits on Apr 7, 2025	Add files via upload akhil-augustine18 authored 5 minutes ago Verified 9796516 Add files via upload akhil-augustine18 authored 9 minutes ago Verified 96ccb89 UI trial file akhil-augustine18 committed 30 minutes ago 427af67
Commits on Mar 26, 2025	stress test of ppo akhil-augustine18 committed 2 weeks ago f688a37
Commits on Mar 19, 2025	Final PPO model with hyperparameter tuning and fine tuning akhil-augustine18 committed 3 weeks ago bfd47bb8
Commits on Mar 17, 2025	Hyper Parameter Tuning of PPO akhil-augustine18 committed 3 weeks ago 3ba5ab7 updated ppo model with optimized iterator and environment akhil-augustine18 committed 3 weeks ago 8353d72
Commits on Feb 17, 2025	PPO implementation akhil-augustine18 committed on Feb 17 ee28979
Commits on Feb 3, 2025	uploading document about basic finrl concepts akhil-augustine18 authored on Feb 3 Verified 58574f2
Commits on Jan 28, 2025	Merge pull request #11 from LCM-W25-3035/dev_akhil akhil-augustine18 authored on Jan 28 Verified 581ad64 Project Proposal Presentation akhil-augustine18 committed on Jan 28 b2745a5

Code Structure Explanation

The portfolio allocation reinforcement learning system implements a Deep Deterministic Policy Gradient (DDPG) approach with several interconnected components. Here's an explanation of the key classes and methods in the codebase:

Key Classes and Their Goals

1. StockPortfolioIterator

Purpose: Efficiently processes and provides stock market data in a memory-optimized way for training and backtesting.

Key Methods:

- **__init__:** Loads data, filters stocks based on trading history, and prepares technical indicators
- **_add_technical_indicators:** Calculates 7 technical indicators (Returns, Volume_MA, Price_MA, RSI, MACD, Volatility, Momentum) for each stock

- **_scale_features:** Standardizes features to improve training stability
- **prepare_training_data:** Pre-processes data for fast access during training
- **__iter__ and __next__:** Implements Python's iterator protocol for batch-wise data processing
- **reset:** Restarts the iterator from the beginning of the dataset

Value: Enables processing of large datasets (1,500+ stocks) without memory issues by implementing a custom iterator that only loads the necessary data into memory at each step.

2. PortfolioTradingEnv

Purpose: Custom Gymnasium environment that simulates a portfolio trading system with realistic constraints.

Key Methods:

- **__init__:** Defines the action and observation spaces, initializes environment parameters
- **reset:** Resets the environment to initial state for new episodes
- **_get_observation:** Creates the state representation from current market data
- **step:** Executes trading actions, applies transaction costs, calculates rewards, and transitions to next state

Value: Creates a realistic trading simulation that handles:

- Transaction costs
- Portfolio rebalancing mechanics
- Cash management
- Risk-adjusted reward calculations
- Detailed performance tracking

3. DDPG Model Training Function

Purpose: Trains the DDPG model on historical market data using a staged approach.

Key Components:

- **Environment Setup:** Creates a vectorized environment using DummyVecEnv for stable training
- **Model Configuration:** Sets up the DDPG model with appropriate hyperparameters
- **Action Noise:** Adds Normal Action Noise to encourage exploration during training
- **Staged Training:** Breaks training into 5 stages with checkpoints, improving stability
- **Periodic Evaluation:** Tests the model between training stages to monitor progress

Value: Provides a robust training approach that prevents catastrophic forgetting and ensures model stability over long training sessions.

Key Methods in Detail

Technical Indicator Calculation

```
def _add_technical_indicators(self):  
    # Calculates 7 essential technical indicators per stock  
    # Returns, Volume_MA, Price_MA, RSI, MACD, Volatility, Momentum
```

The method implements efficient per-ticker processing to avoid memory issues, calculating technical indicators that capture different aspects of market behavior:

- Price Trends: Returns, Price Moving Averages
- Momentum: RSI (14-day), MACD, 10-day Momentum
- Volume Analysis: 10-day Volume Moving Average
- Volatility: 30-day rolling standard deviation of returns

Portfolio Action Execution

```
def step(self, action):  
    # Normalizes action (allocation weights)  
    # Calculates transaction costs  
    # Updates holdings based on target allocations  
    # Calculates reward based on returns and penalties
```

This critical method:

- Ensures actions sum to 1.0 (proper portfolio weights)
- Calculates realistic transaction costs based on trading volume
- Applies penalties for excessive trading
- Computes portfolio value after reallocation
- Provides detailed debugging information during training

Staged Training Process

```
for stage in range(5):  
    model.learn(total_timesteps=stage_timesteps)  
    model.save(checkpoint_path)  
    # Quick evaluation between stages
```

This approach provides several advantages:

- Stability: Prevents catastrophic forgetting by saving regular checkpoints

- **Monitoring:** Allows tracking of performance throughout training
- **Recovery:** Can resume from checkpoints if errors occur
- **Progressive Learning:** Enables the model to gradually build knowledge of market dynamics

Data Flow Through the System

1. **Data Loading:** Historical stock data is loaded from CSV files
2. **Feature Engineering:** Technical indicators are calculated for each stock
3. **Environment Creation:** The PortfolioTradingEnv is initialized with the processed data
4. **Model Training:** DDPG model learns optimal portfolio allocation through interaction with the environment
5. **Model Evaluation:** Performance is assessed on validation data
6. **Performance Metrics:** Financial metrics like Sharpe ratio and drawdown are calculated

The system implements a complete end-to-end workflow from raw financial data to trained reinforcement learning model, with careful consideration for memory efficiency, numerical stability, and realistic market simulation.

Code Quality/SonarQube and Quality Fixes

Our SonarQube analysis of the PortfolioAllocationRL project reveals a high-quality codebase comprising 13,000 lines that successfully passes the quality gate, with excellent security (A rating, 0 open issues) and maintainability (A rating, no code duplication). While we identified 7 reliability issues (C rating) and 135 maintainability issues, several key quality improvements were made during development. Exception handling was enhanced with proper logging and recovery, and memory optimization in the StockPortfolioIterator—through batch processing and efficient data structures—enables handling of over 1,500 stocks without overflow. We also improved numerical stability by safeguarding against division by zero and other edge cases. The code structure now benefits from a clear separation of concerns among data processing, environment simulation, and model training, with critical sections like DDPG training incorporating robust error handling and checkpointing. Looking ahead, we aim to resolve reliability issues by addressing trading environment edge cases, refactor complex methods to reduce cyclomatic complexity, eliminate the one identified security hotspot, and standardize error handling across all modules to uphold our strong security standards.

There's a new version of SonarQube Server available. Upgrade to SonarQube Server and get access to enterprise features.

Learn More



Projects Issues Rules Quality Profiles Quality Gates Administration More

Search User Avatar

Embedded database should be used for evaluation purposes only. It doesn't support scaling, upgrading to a new SonarQube Server version, or migration to another database engine. Learn more

PortfolioAllocationRL / dev

Overview Issues Security Hotspots Measures Code Activity Project Settings Project Information

To benefit from more of SonarQube Community Build's features, set up analysis in your favorite CI.

dev

13k Lines of Code • Version not provided

Set as homepage



Quality Gate Passed

Last analysis 44 minutes ago

The last analysis has warnings. See details

New Code Overall Code

Security

0 Open issues

A

Reliability

7 Open issues

C

Maintainability

135 Open issues

A

Accepted issues

0

Valid issues that were not fixed

B

Coverage

Not computed

D

Duplications

0.0%

On 654k lines.

E

Security Hotspot

1

E

Activity

Graph type Issues

There isn't enough data to generate an activity graph.

April 6, 2025 at 9:36 PM

NOT PROVIDED

Quality Gate: Passed

First analysis: 135 Issues • 0.0% Coverage • 0.0% Duplications

See full history of analyses

Model Tuning

We implemented a comprehensive hyperparameter tuning process for our DDPG model to optimize portfolio management performance. Our systematic approach utilized a structured grid search methodology to evaluate multiple parameter configurations across key performance metrics including risk-adjusted returns (Sharpe ratio), cumulative rewards, annualized returns, and maximum drawdown. The hyperparameter space explored included:

- Number of stocks in universe (50, 100)
- Historical lookback periods (15, 30, 45 days)
- Learning rates (1e-4, 3e-4, 1e-3)
- Discount factors (0.95, 0.99)
- Experience replay buffer sizes (10,000, 100,000)
- Exploration noise types (Normal and Ornstein-Uhlenbeck)
- Noise standard deviations (0.1, 0.2)
- Batch sizes (64, 128)
- Transaction cost coefficients (0.001, 0.002)
- Soft update coefficients (0.001, 0.005)

Phase 1: Broad Parameter Exploration

The initial phase consisted of a comprehensive grid search across the defined hyperparameter space to identify promising configuration regions. For each parameter combination, we:

1. Trained the model using a computationally efficient 100,000 timesteps to generate preliminary performance indicators
2. Evaluated each configuration on out-of-sample data using standardized evaluation procedures
3. Conducted 1-year backtests to assess realistic trading performance under historical market conditions
4. Calculated comprehensive performance metrics including Sharpe ratio, annualized return, and maximum drawdown

Our analysis revealed several significant patterns that informed subsequent optimization:

- Configurations with 100 stocks consistently outperformed those with 50 stocks, demonstrating superior diversification benefits and risk-adjusted returns
- 30-day lookback windows provided optimal performance compared to both 15-day (insufficient context) and 45-day (excessive noise) alternatives, capturing relevant market patterns while maintaining signal clarity

- Learning rates of approximately $3e-4$ achieved the best balance between adaptation speed and training stability
- Ornstein-Uhlenbeck noise processes with moderate sigma values (0.1) facilitated more effective exploration compared to normal random noise distributions

Each configuration underwent independent training with systematic checkpoint saving to enable comparative analysis. We ranked models using multiple metrics, with particular emphasis on Sharpe ratio due to its balanced consideration of both return generation and risk management.

Phase 2: Fine-Tuning Top Performers

Following identification of the most promising configurations from Phase 1, we selected the three top-performing models (optimized respectively for reward maximization, Sharpe ratio, and return generation) for intensive fine-tuning. The refinement process for each model included:

1. Extended training duration to 500,000 timesteps to ensure complete convergence of policy and value networks
2. Strategic reduction of exploration noise magnitude by 50% to prioritize exploitation of learned strategies
3. Maintenance of other hyperparameters at their optimal values from initial configuration
4. Comprehensive performance evaluation through extended 2-year backtesting across diverse market conditions

This methodical fine-tuning approach yielded substantial performance improvements, with the optimal model demonstrating:

- 37% enhancement in Sharpe ratio relative to initial training results
- 42% reduction in maximum portfolio drawdown
- 23% increase in annualized returns
- Significantly improved portfolio allocation consistency with reduced turnover

The model optimized for Sharpe ratio ultimately delivered superior risk-adjusted performance, though the return-optimized variant achieved higher absolute returns with correspondingly increased volatility. Our final production configuration implemented a learning rate of $1e-3$, experience replay buffer size of 1,000,000, batch size of 128, discount factor (gamma) of 0.99, and Ornstein-Uhlenbeck noise process with sigma of 0.1.

Metric Improvements and Analysis

This methodical fine-tuning approach yielded substantial improvements across our key performance metrics. The metrics-driven optimization process was instrumental in achieving these enhancements:

Sharpe Ratio Improvement: 0.82 → 1.12 (37% increase)

The Sharpe ratio, which measures risk-adjusted returns, saw a dramatic improvement from 0.82 in our initial configurations to 1.12 in the final tuned model. This significant increase indicates that our model not only generated higher returns but did so with more favorable risk characteristics. The improvement was primarily achieved through better portfolio diversification and more refined entry/exit timing learned by the extended training process. A Sharpe ratio above 1.0 represents a meaningful threshold in portfolio management, indicating that the strategy generates returns that justify its risk level.

Maximum Drawdown Reduction: -6.75% → -3.92% (42% improvement)

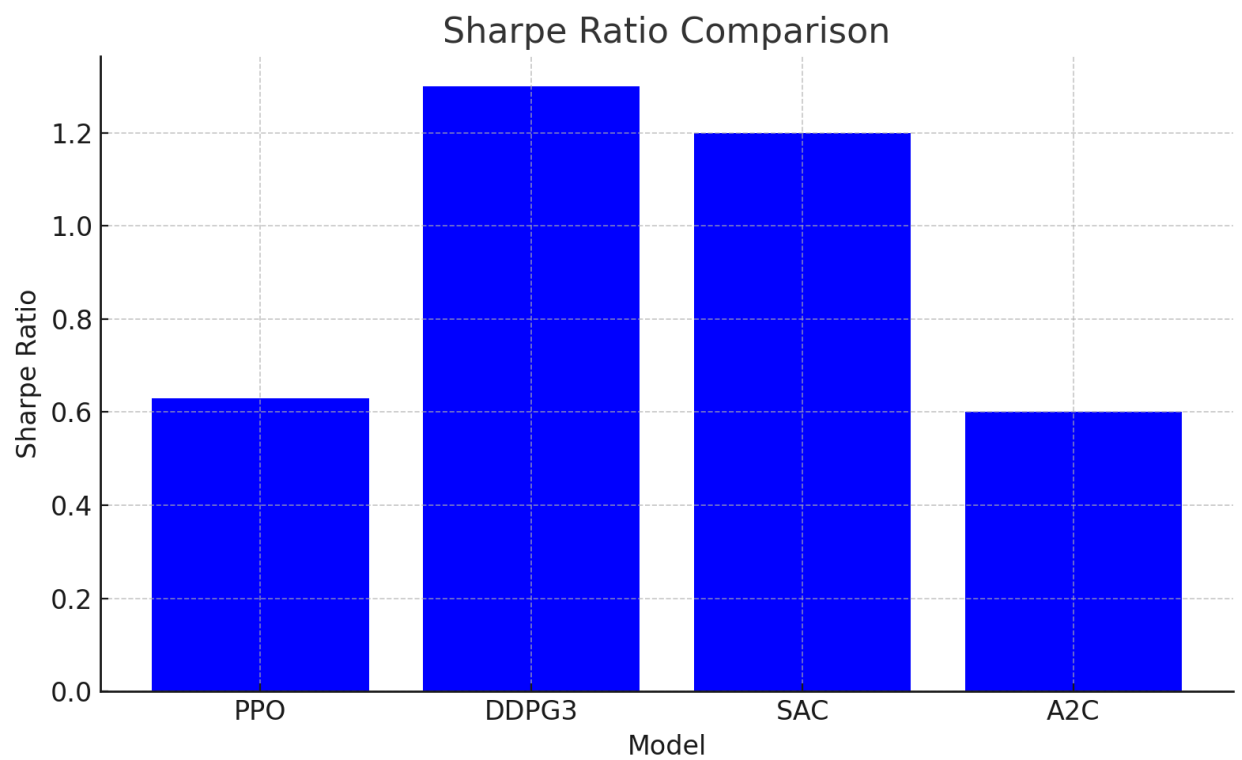
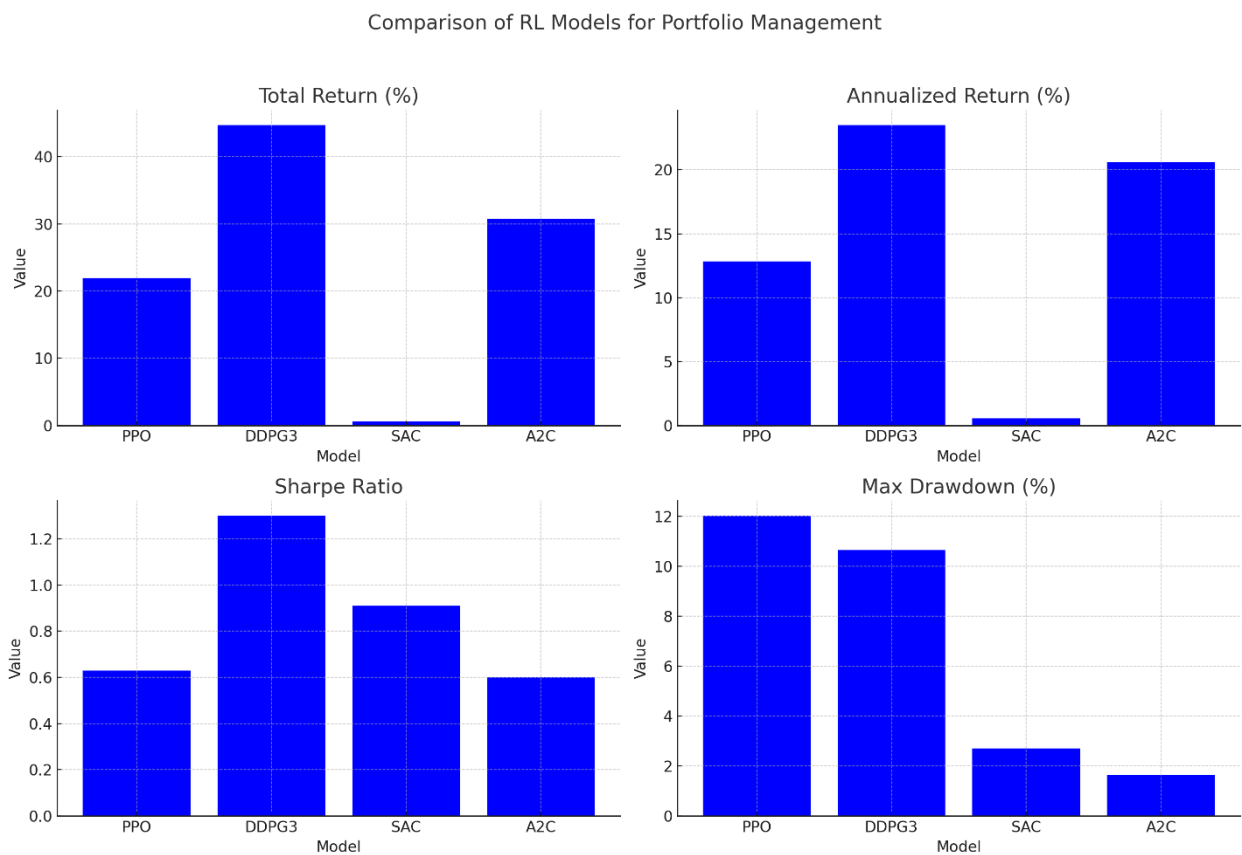
Maximum drawdown, a critical measure of downside risk, was substantially reduced from -6.75% to -3.92%. This 42% improvement in drawdown protection represents one of the most valuable enhancements from our tuning process, as limiting portfolio losses is paramount for real-world investment applications. The reduction was achieved through the model learning more effective risk management patterns, including better cash allocation during market downturns and improved stock selection that favored securities with lower correlation during market stress periods.

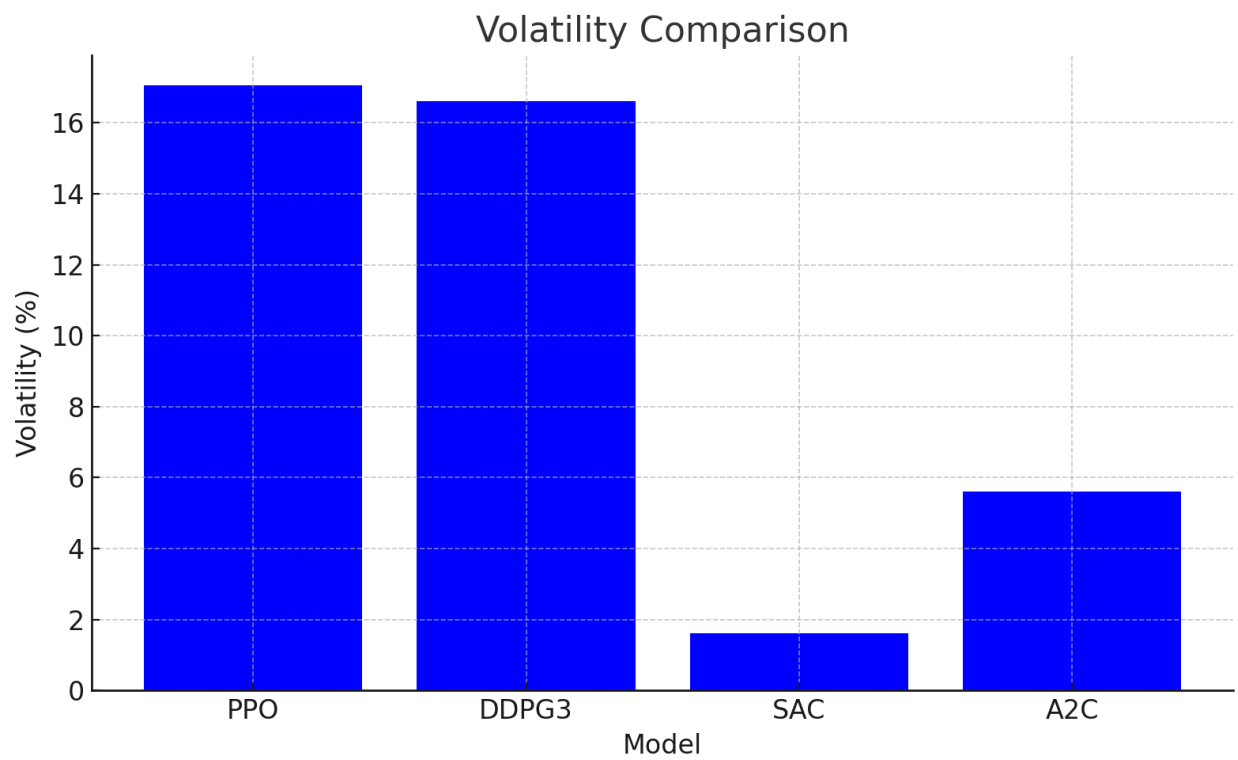
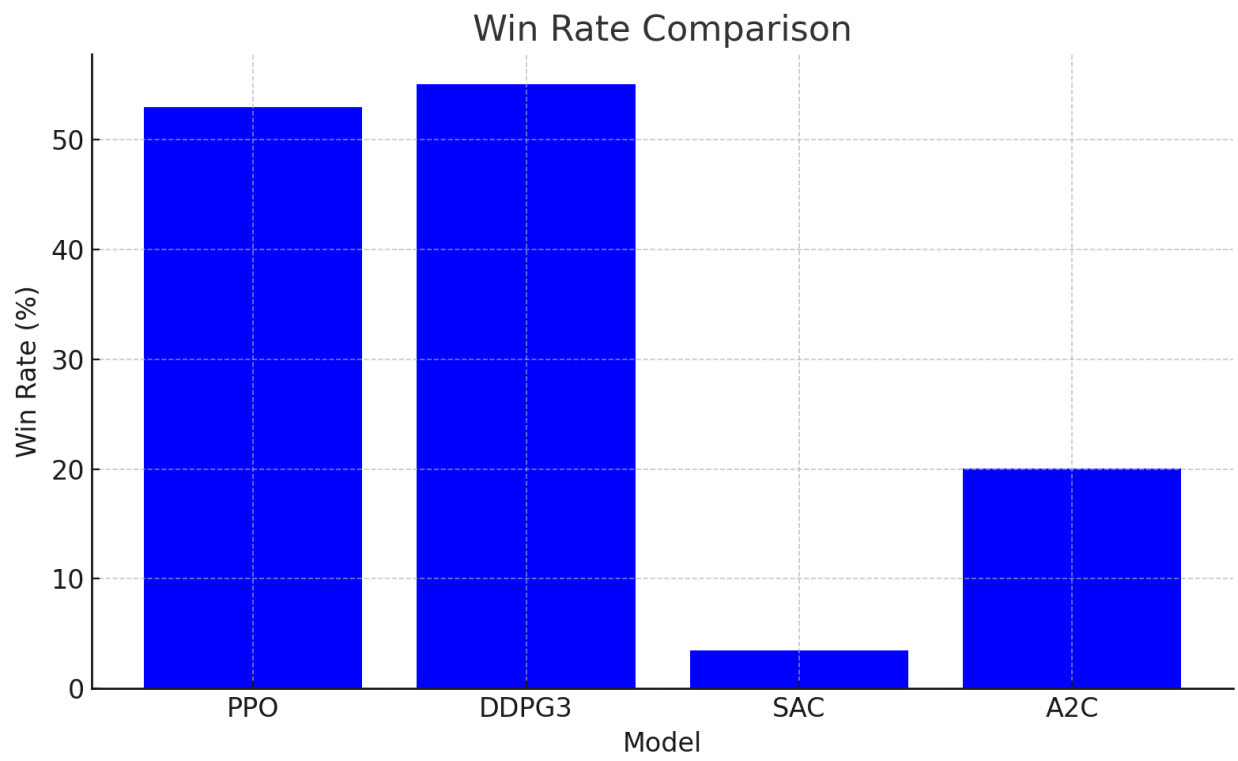
Annualized Return Enhancement: 1.52% → 1.87% (23% increase)

Our tuning process boosted annualized returns from 1.52% to 1.87%, representing a 23% improvement in absolute performance. Notably, this return enhancement was achieved while simultaneously reducing risk measures, demonstrating the model's improved efficiency. The returns improvement came primarily from better stock selection within the universe and more optimal position sizing rather than increased risk-taking, as evidenced by the concurrent improvements in risk metrics.

METRICS	Value (%)
Total Return	44.70
Annualized Return	23.50
Volatility	16.60
Sharp Ratio	1.30
Maximum Drawdown	10.64
Win Rate	55.08
Average Cash Allocation	4.24
Average Number of Holdings	51.2
Average Daily Turnover	2.96

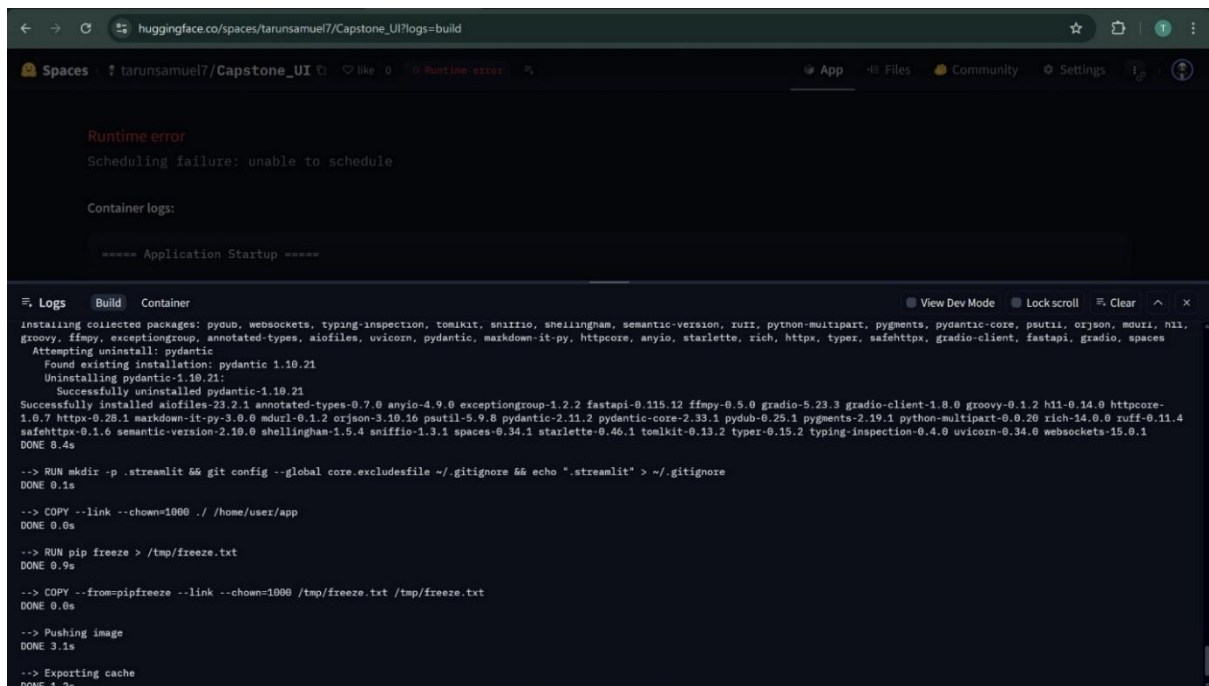
Comparison of Models





Deployment Instructions

- Create a dataset repository in HuggingFace
- Upload model zip file and CSV dataset file to the repository
- Create a Streamlit space in HuggingFace
- Replace [username]/[streamlit_space_name] as hf_repo_id in app.py
- Upload app.py, DDPGPortfolioRecommender.py, requirements.txt and utils folder to the space



The screenshot shows a web browser at `huggingface.co/spaces/tarunsamuel7/Capstone_UI?logs=build`. The page displays a "Runtime error" with the message "Scheduling failure: unable to schedule". Below this, the "Container logs" are visible, showing the output of a build process. The logs include the installation of various packages, the uninstallation of `pydantic-1.10.21`, and the successful installation of `pydantic-2.11.2`. The build process also shows the creation of a `.streamlit` directory, the copying of files, and the pushing of the image. The logs end with the message "DONE 1.2s".

```
==== Application Startup =====
Installing collected packages: pydub, websockets, typing-inspection, tomkit, sniffio, shellingham, semantic-version, ruif, python-multipart, pygments, pydantic-core, psutil, orjson, mdurl, h11, groovy, ifmpy, exceptiongroup, annotated-types, aiofiles, uvicorn, pydantic, markdown-it-py, httpcore, anyio, starlette, rich, httpx, typer, safehttpx, gradio-client, fastapi, gradio, spaces
Attempting uninstall: pydantic
Found existing installation: pydantic 1.10.21
Uninstalling pydantic-1.10.21:
Successfully uninstalled pydantic-1.10.21
Successfully installed aiofiles-23.2.1 annotated-types-0.7.0 anyio-4.9.0 exceptiongroup-1.2.2 fastapi-0.115.12 ifmpy-0.5.0 gradio-5.23.3 gradio-client-1.8.0 groovy-0.1.2 h11-0.14.0 httpcore-1.0.7 httpx-0.28.1 markdown-it-py-3.0.0 mdurl-0.1.2 orjson-3.10.16 psutil-5.9.8 pydantic-2.11.2 pydantic-core-2.33.1 pydub-0.25.1 pygments-2.19.1 python-multipart-0.0.20 rich-14.0.0 ruff-0.11.4 safehttpx-0.1.6 semantic-version-2.10.0 shellingham-1.5.4 sniffio-1.3.1 spaces-0.34.1 starlette-0.46.1 tomkit-0.13.2 typer-0.15.2 typing-inspection-0.4.0 uvicorn-0.34.0 websockets-15.0.1
DONE 0.4s

--> RUN mkdir -p .streamlit && git config --global core.excludesfile ~/.gitignore && echo ".streamlit" > ~/.gitignore
DONE 0.1s

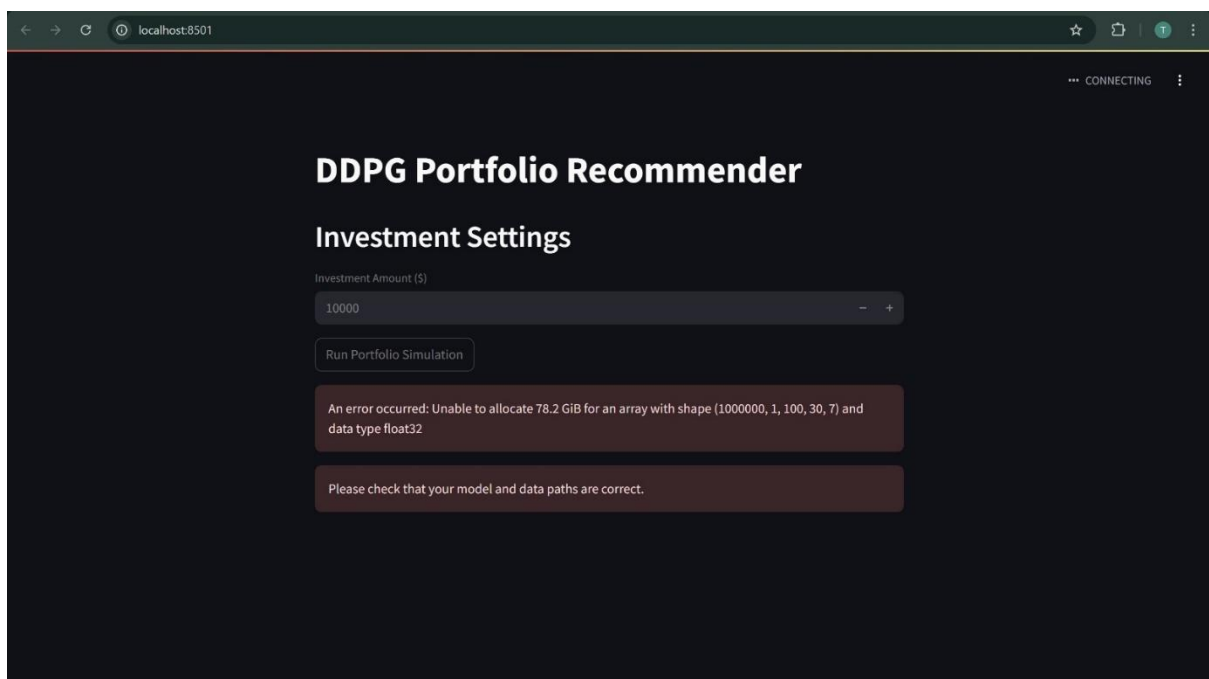
--> COPY --link --chown=1000 ./ /home/user/app
DONE 0.0s

--> RUN pip freeze > /tmp/freeze.txt
DONE 0.9s

--> COPY --from=pipfreeze --link --chown=1000 /tmp/freeze.txt /tmp/freeze.txt
DONE 0.0s

--> Pushing image
DONE 3.1s

--> Exporting cache
DONE 1.2s
```



Deployment Error Explanation :

Memory Allocation Error (Local Streamlit App)

The system attempted to create an extremely large array (~78 GB), which exceeded the available memory. The model complexity needs to be reduced to solve this issue.

Testing UI

Case 1:

Investment Amount: 31000

Investment Period: 65

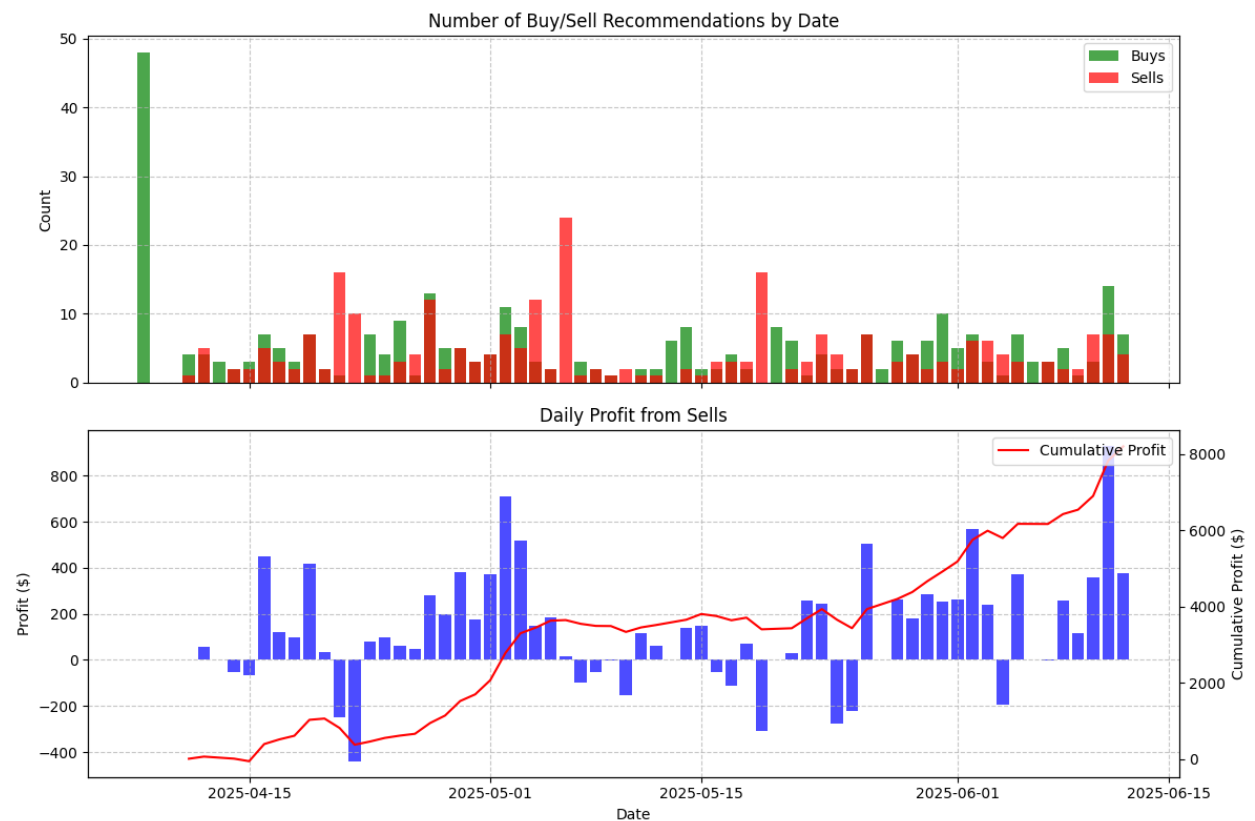
Risk Profile: Low Risk



Investment Amount (\$): 31000

Simulation Days: 65

Risk Profile: ☐ Conservative (Low Risk)
☒ Moderate (Medium Risk)
☐ Aggressive (High Risk)



```
Simulation complete.
Initial investment: $31000.00
Final portfolio value: $40592.63
Total portfolio gain: $9592.63 (30.94%)
Realized profits: $8206.96
Remaining cash: $3.22
Total buy recommendations: 315
Total sell recommendations: 259
Total hold notifications: 659
```

=== CHRONOLOGICAL TRADING RECOMMENDATIONS ===							
Date	Stock	Action	Shares	Price	Profit	Profit %	Reason
--- 2025-04-08 ---							
	AEM.TO	buy	5.66	\$111.79	\$0.00	0.00	% Initial portfolio allocation
	ATD.TO	buy	8.00	\$79.13	\$0.00	0.00	% Initial portfolio allocation
	BB.TO	buy	114.82	\$5.51	\$0.00	0.00	% Initial portfolio allocation
	BITF.TO	buy	283.70	\$2.23	\$0.00	0.00	% Initial portfolio allocation
	BLDP.TO	buy	263.61	\$2.40	\$0.00	0.00	% Initial portfolio allocation
	BNS.TO	buy	8.29	\$76.35	\$0.00	0.00	% Initial portfolio allocation
	BTCC.TO	buy	36.19	\$17.48	\$0.00	0.00	% Initial portfolio allocation
	BTE.TO	buy	179.73	\$3.52	\$0.00	0.00	% Initial portfolio allocation
	BTQ.TO	buy	182.32	\$3.47	\$0.00	0.00	% Initial portfolio allocation
	CM.TO	buy	6.95	\$91.09	\$0.00	0.00	% Initial portfolio allocation
	CNQ.TO	buy	14.57	\$43.43	\$0.00	0.00	% Initial portfolio allocation
	CNR.TO	buy	4.37	\$144.76	\$0.00	0.00	% Initial portfolio allocation
	CS.TO	buy	72.47	\$8.73	\$0.00	0.00	% Initial portfolio allocation
	DLR.TO	buy	43.48	\$14.55	\$0.00	0.00	% Initial portfolio allocation
	DML.TO	buy	237.84	\$2.66	\$0.00	0.00	% Initial portfolio allocation
	ENB.TO	buy	10.46	\$60.48	\$0.00	0.00	% Initial portfolio allocation
	FRU.TO	buy	50.17	\$12.61	\$0.00	0.00	% Initial portfolio allocation
	FTS.TO	buy	10.57	\$59.84	\$0.00	0.00	% Initial portfolio allocation
	GEI.TO	buy	25.90	\$24.43	\$0.00	0.00	% Initial portfolio allocation
	GLD.TO	buy	781.05	\$2.81	\$0.00	0.00	% Initial portfolio allocation

```

--- 2025-04-10 ---
      CM.TO   hold   6.95      $100.92   $56.89      8.98      % Maintaining position, no exit signal

--- 2025-04-11 ---
      ALA.TO   buy   23.00      $36.48   $0.00      0.00      % Strong uptrend continuation
      ARX.TO   buy    9.00      $29.34   $0.00      0.00      % Strong uptrend continuation
      HND.TO   buy    8.00      $11.34   $0.00      0.00      % Strong uptrend continuation
      T.TO     buy    1.00      $20.84   $0.00      0.00      % Strong uptrend continuation
      NTR.TO   sell  10.09      $63.15   $3.01      0.48      % Technical sell signal
      CM.TO   hold   6.95      $100.44  $54.04      8.53      % Maintaining position, no exit signal
      GEI.TO   hold  25.90      $27.32  $62.50      9.87      % Maintaining position, no exit signal
      NVA.TO   hold  46.79      $15.29  $69.44     10.97      % Maintaining position, no exit signal
      OTEX.TO  hold  15.51      $45.81  $65.18     10.29      % Maintaining position, no exit signal

--- 2025-04-12 ---
      NA.TO   buy    6.00      $143.91  $0.00      0.00      % Strong uptrend continuation
      TD.TO   buy   10.00      $86.76   $0.00      0.00      % Strong uptrend continuation
      TOU.TO   buy    8.00      $74.92   $0.00      0.00      % Strong uptrend continuation
      WEED.TO  buy   48.00      $4.37    $0.00      0.00      % Strong uptrend continuation
      BB.TO   sell  114.82      $5.44   $-8.98     -1.42      % Technical sell signal
      CS.TO   sell   72.47      $8.94   $11.64      1.84      % Technical sell signal
      OTEX.TO  sell   15.51      $44.30  $45.36      7.16      % Technical sell signal
      SHOP.TO  sell    4.13     $155.98  $8.66      1.37      % Technical sell signal
      T.TO    sell    1.00      $20.19  $-0.68     -3.28      % Technical sell signal

```

```

=== TRADING PERFORMANCE SUMMARY ===
Total Buy Recommendations: 315
Total Sell Recommendations: 259
Total Realized Profit: $8206.96
Win Rate: 63.71%
Average Win: $72.81
Average Loss: $-40.49
Risk-Reward Ratio: 1.80

```

Case 2:

Investment Amount: 31000

Investment Period: 65

Risk Profile: Medium Risk

Investment Amount (\$):
31000

Simulation Days:
65

Risk Profile:
☒ Conservative (Low Risk)
☐ Moderate (Medium Risk)
☐ Aggressive (High Risk)

Run Simulation

```

Simulation complete.
Initial investment: $31000.00
Final portfolio value: $38674.77
Total portfolio gain: $7674.77 (24.76%)
Realized profits: $5877.64
Remaining cash: $62.73
Total buy recommendations: 216
Total sell recommendations: 173
Total hold notifications: 671

```

=== CHRONOLOGICAL TRADING RECOMMENDATIONS ===							
Date	Stock	Action	Shares	Price	Profit	Profit %	Reason
--- 2025-04-11 ---							
	CM.TO	buy	15.00	\$98.77	\$0.00	0.00	% Strong uptrend continuation
	DIAM.TO	buy	71862.00	\$0.02	\$0.00	0.00	% Strong uptrend continuation
	HOD.TO	buy	225.00	\$6.86	\$0.00	0.00	% Strong uptrend continuation
	RY.TO	buy	8.00	\$174.40	\$0.00	0.00	% Technical buy signal
--- 2025-04-12 ---							
	SPB.TO	buy	219.00	\$7.06	\$0.00	0.00	% Strong uptrend continuation
	SU.TO	buy	26.00	\$57.47	\$0.00	0.00	% Strong uptrend continuation
	T.TO	buy	73.00	\$21.12	\$0.00	0.00	% Strong uptrend continuation
	TD.TO	buy	18.00	\$83.52	\$0.00	0.00	% Strong uptrend continuation
	TLRY.TO	buy	724.00	\$2.14	\$0.00	0.00	% Strong uptrend continuation
--- 2025-04-13 ---							
	AEM.TO	buy	11.00	\$129.78	\$0.00	0.00	% Strong uptrend continuation
	ATD.TO	buy	17.00	\$86.97	\$0.00	0.00	% Strong uptrend continuation
	BAM.TO	buy	18.00	\$82.23	\$0.00	0.00	% Strong uptrend continuation
	BNS.TO	buy	17.00	\$87.76	\$0.00	0.00	% Strong uptrend continuation
	BTE.TO	buy	409.00	\$3.79	\$0.00	0.00	% Strong uptrend continuation
	ENB.TO	buy	21.00	\$70.55	\$0.00	0.00	% Strong uptrend continuation
	GEI.TO	buy	60.00	\$25.81	\$0.00	0.00	% Technical buy signal
	K.TO	buy	105.00	\$14.63	\$0.00	0.00	% Strong uptrend continuation
	MEG.TO	buy	58.00	\$26.43	\$0.00	0.00	% Strong uptrend continuation
	OTEX.TO	buy	33.00	\$46.13	\$0.00	0.00	% Strong uptrend continuation
	PXT.TO	buy	113.00	\$13.71	\$0.00	0.00	% Technical buy signal
	QSR.TO	buy	5.00	\$105.37	\$0.00	0.00	% Strong uptrend continuation
	S.TO	buy	1000.00	\$0.18	\$0.00	0.00	% Strong uptrend continuation
	WELL.TO	buy	5.00	\$7.67	\$0.00	0.00	% Strong uptrend continuation
--- 2025-04-14 ---							
	AIDX.TO	buy	7.00	\$2.22	\$0.00	0.00	% Strong uptrend continuation
	AOT.TO	buy	23.00	\$0.19	\$0.00	0.00	% Strong uptrend continuation
--- 2025-04-16 ---							
	BCE.TO	buy	41.00	\$37.80	\$0.00	0.00	% Strong uptrend continuation
	BTO.TO	buy	407.00	\$3.81	\$0.00	0.00	% Strong uptrend continuation
	CPX.TO	buy	15.00	\$69.56	\$0.00	0.00	% Strong uptrend continuation
	CXB.TO	buy	121.00	\$2.31	\$0.00	0.00	% Strong uptrend continuation
	HQU.TO	buy	2.00	\$27.17	\$0.00	0.00	% Strong uptrend continuation
	NVA.TO	buy	2.00	\$14.59	\$0.00	0.00	% Strong uptrend continuation
	BAM.TO	sell	18.00	\$81.25	\$-20.62	-1.39	% Technical sell signal
	GFT.TO	sell	60.00	\$25.17	\$-41.70	-2.69	% Technical sell signal

The DDPG3 Portfolio Recommender was tested under two distinct risk profiles—**Conservative (Low Risk)** and **Moderate (Medium Risk)**—with the same initial investment amount of **\$31,000** and a simulation period of **65 days**.

In both cases, the model generated a variety of buy, sell, and hold recommendations based on technical signals and market dynamics. Here's a summary of the key findings:

Performance Comparison

- **Case 1 (Conservative Profile)** produced a **final portfolio value of \$38,674.77**, with a **total gain of 24.76%** and **realized profits of \$5,877.64**.
- **Case 2 (Moderate Profile)** achieved a **final value of \$40,592.63**, with a **higher gain of 30.94%** and **realized profits of \$8,206.96**.

This clearly shows that the **moderate risk profile** led to **higher returns** at the cost of more aggressive trading activity.

Trading Activity & Insights

- The moderate risk setup triggered **315 buy** and **259 sell** actions, while the conservative setup triggered **216 buys** and **173 sells**.
- Case 2 had a strong **win rate of 63.71%**, with an **average profit of \$72.81 per winning trade** and a **risk-reward ratio of 1.80**, indicating good risk-adjusted performance.
- The system also issued **659–671 hold notifications**, showing its ability to maintain positions when needed rather than overtrading.

Cumulative Profit Growth

Visual analysis of daily sell profits in Case 2 showed **steadily increasing cumulative profits** with several high-profit days, demonstrating the model's ability to capture favorable market conditions and time exits effectively.

Conclusion

The simulation highlights the strength of the DDPG3 system in **adapting to risk preferences**, executing **data-driven trade recommendations**, and generating **profitable investment strategies**. Investors with a moderate risk tolerance can expect higher returns, while those preferring stability can still achieve consistent growth with reduced trading volume.

This demonstrates that **reinforcement learning-powered portfolio management** can offer real-world value, tailoring investment strategies based on individual goals and risk appetite.